

CSC490: Assignment 2

Team Members

Name	Student ID
Aarya Prakash	1008790257
Derek Huynh	1008849052
Merrick Liu	1008951920
Rhys Balevicius	1000801974

University of Toronto
Department of Computer Science

1 Product Overview

We are building Lighthouse a code search engine exposed as an MCP that improves the performance of AI coding agents by surfacing better context. For our initial implementation, we will index GitHub repositories that the user selects.

Here is a quick walk-through of how a developer would use Lighthouse:

1. Developer selects GitHub repositories/branch that they want indexed.
2. Lighthouse indexes the repositories.
3. Developer adds Lighthouse MCP to their AI coding agent.
4. While the agent is working, if it decides it needs context, it will search for context using the Lighthouse MCP.

At a high level, Lighthouse has 3 key components that we will discuss in the remainder of this report:

1. Ingesting GitHub repositories / code bases.
2. Querying our search index.
3. Evaluating the performance of our search index.

In terms of production deployments, there are useage patterns and requirements (latency and traffic) for data ingestion and retrieval we need to be aware of:

- **Ingestion** can be broken into 2 cases. The first is the initial indexing of repositories and the second is the re-indexing / maintenance of the index as code is modified. Both **do not have low latency requirements** and will have much **less traffic** compared to retrieval as these are background tasks. Traffic will be **relatively predictable** as the vast majority of engineers work similar hours.
- **Retrieval/Querying** will be **high traffic** and **require low latency** as our search tool will be constantly during code development. Traffic will be **relatively predictable** as the vast majority of engineers work similar hours.

These requirements will be important as we develop and make design decisions.

2 Aspirational Datasets

Our goal is to improve the performance of AI coding agents which may produce incorrect changes because they lack relevant context that exists outside their immediate working scope.

To build and evaluate the retrieval quality of our system, the ideal dataset would consist of a coding task with a point-in-time snapshot of the full software environment: the target repository, its direct dependencies, documentation, and version history along with a gold label identifying the precise, minimal set of non-local information an agent would have needed to produce a correct change. This label is the core artifact: it answers the question “*what did the agent need to know, but didn’t, in order to get this right?*” and it spans multiple axes of context, including interface contracts and type signatures across module and library boundaries, downstream callers and invariants that a local change might violate, architectural conventions and design rationale recoverable from project history, relevant documentation or specifications from external dependencies, and code snippets.

Each entry would additionally include a failed agent attempt (produced without sufficient context), the provided context, the corresponding corrected code (produced by a human or by an agent given adequate context), and the failure signal linking the two such as a test failure, type error, or behavioural regression. The causal structure of this data is what makes it powerful: the delta between the failed and successful attempts is attributable to the presence or absence of the labelled context. With such a dataset, one could train and evaluate any retrieval or reasoning approach against a ground-truth definition of what “the right context” is for a given task, and measure whether surfacing it actually changes agent outcomes.

Note: nested type definitions are provided in [Appendix A](#).

Below is a primary dataset along with supporting secondary datasets. The primary dataset provides our most important pieces of data while the secondary datasets provide more useful coding context we can use.

Primary Dataset: Agent Failures/Successes with Context

Each row captures a coding task where an agent failed or succeeded, the corrected outcome, and a precise attribution of what context was missing/used. On a successful completion of a task, all cells related to failure will be null. This dataset is crucial as it directly measures what context needs to be surfaced along with the changes required by the coding agent.

Task

Field	Type	Notes
repo	str	Target repository
repo_snapshot	commit_sha	Frozen state of repo at task time
dependency_snapshot	[package@version]	Pinned versions of all transitive dependencies
task	str	Natural language description of the coding task
success	Boolean	Did the agent succeed
agent_attempt	AgentAttempt	Metadata of attempt
failure_signal	FailureSignal	Failure manifestation
corrected_diff	Diff	Correct change
context_used	[Context]	Context used on attempt
missing_context	[Context]	Required unseen context

Secondary Dataset: Interface Contracts and Cross-Boundary Specifications

Each row describes a function or module boundary, its (possibly implicit) contract, and the set of callers/consumers that depend on that contract across repo and library boundaries.

Function

Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
function	fully_qualified_name	Canonical name
defined_in	{file, span}	Definition location
contract	Contract	Behavioural specification
consumers	[Consumer]	Call sites
known_violations	[KnownViolation]	Historical breaks

Secondary Dataset: Architectural Conventions and Project Norms

Each row encodes an implicit or explicit convention in a codebase, examples of code that follow it, and examples of agent-generated code that violates it.

ConventionSchema

Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
convention	Convention	Normative pattern
conforming_examples	[{file, span}]	Correct usages
violations	[Violation]	Broken instances

Secondary Dataset: Design Rationale and Temporal Context

Each row links a region of code to the historical decisions that explain *why* it is the way it is: the context an agent needs to avoid "cleaning up" intentional complexity.

CodeHistory

Field	Type	Notes
repo	str	Repository
repo_snapshot	commit_sha	Frozen state
code_region	{file, span}	Location
current_form	str	Present code
history	[HistoryEntry]	Prior commits
rationale	Rationale	Design justification
naive_refactors	[NaiveRefactor]	Tempting but incorrect changes

Secondary Dataset: Cross-Repository Dependency Context

Each row captures a usage of an external library and the external documentation, source, or changelog context needed to use it correctly.

DependencyUsage

Field	Type	Notes
consumer_repo	str	Consuming repo
repo_snapshot	commit_sha	Frozen state
dependency	{name, version}	External package
usage_location	{file, span}	Call site
api_used	fully_qualified_name	Referenced symbol
required_context	[RequiredContext]	External knowledge
version_sensitivity	VersionSensitivity	Version constraints
misuse_examples	[MisuseExample]	Incorrect usages

3 Reality Check

The datasets above ask for five things per task: a task statement, a point-in-time code/dependency snapshot, a failure signal, a corrected change, and explicit attribution of the missing non-local context. No single public source gives all five, so the practical path is to combine (1) existing benchmarks, (2) mined web data, and (3) synthetic generation pipelines.

Dataset	Description	Commentary
CodeSearchNet	code-docstring pairs across six languages (Python, Java, JS, PHP, Go, Ruby), originally released for training code search models.	Training data for embedding models. Provides high-quality code+NL pairs for contrastive learning (e.g., Milvus semantic search). Also useful for evaluating NL→code retrieval performance.
CodeRAG-Bench	Comprehensive benchmark for retrieval-augmented code generation, covering eight diverse coding tasks (e.g., HumanEval, SWE-Bench, ODEX) with retrieval corpora from five sources: competition solutions, tutorials, library documentation, Stack-Overflow posts, and GitHub repositories.	Evaluation of retrieval quality and RAG pipelines for code generation. Useful for benchmarking how well retrievers surface relevant context (documentation, solutions, posts) to improve code generation across general, open-domain, and repository-level tasks. Provides ground-truth document annotations enabling reliable retrieval evaluation alongside execution-based correctness metrics.
SWE-Bench	2,294 task instances from 12 Python repos. Each instance includes issue, PR, and test patch.	Evaluation benchmark of full agent loop. Run agents with/without MCP service and measure resolution-rate delta.
SWE-Bench Pro	1,865 issues requiring multi-file edits (approx. 107 lines changed). Similar to SWE-Bench except harder.	Same as SWE-Bench. Uses “fail-to-pass” and “pass-to-pass” tests.
SWE-bench-Live	Continuously updated variant of SWE-bench with more recent issues post-training-cutoff. Same schema (issue, PR, test patch).	Anti-contamination eval set. Prevents inflated scores from benchmark leakage into foundation model training data. Use as a held-out evaluation set once our system is tuned on original SWE-bench.
SWE-smith	Toolkit for generating synthetic SWE-bench-style instances via automated code transformations (bug injection, context removal). Can produce instances at scale on arbitrary repos.	Scale up evaluation + training data. Original SWE-bench is only 2K instances; SWE-smith enables orders-of-magnitude more. Useful for stress-testing retrieval across diverse repo structures and for generating Dataset 1 (context-attributed failures) synthetically with known ground-truth context.
SWE-Gym-Raw	Execution environments for SWE-bench instances. Provides Docker setups with pinned dependencies for running tests against agent patches.	Evaluation infrastructure (not a dataset). Enables reproducible eval harness execution and avoids building custom sandboxes. Low priority for data ingestion; high priority for evaluation.
SWE-rebench	Re-evaluated SWE-bench leaderboard with stricter pass/fail criteria (avoids false positives from flaky tests, overcounting, etc.).	Meta-resource for evaluation calibration. Not a dataset to ingest. Use their methodology to ensure our eval harness produces trustworthy results. Reference only.

Dataset	Description	Commentary
RepoBench	Benchmark for repo-level code completion. Tasks require cross-file context (imports, related functions in other files) to complete a function body correctly. 11K instances across Python and Java.	Directly relevant for code graph retrieval evaluation. Tests whether the system surfaces correct cross-file context (e.g., <code>get_callers</code> , <code>get_contract</code>). Strong eval set for the indexing layer independent of the full MCP pipeline. Covers Dataset 2 (contracts/cross-boundary specs).
The Stack v2	67.5 TB of permissively licensed source code from Software Heritage, spanning hundreds of languages.	Training resource for embeddings (not an eval dataset). Suitable for training or fine-tuning code embeddings (e.g., Milvus semantic search). Also useful for large-scale convention mining (Dataset 4). Heavy to process; use targeted subsets.
Software Heritage Graph	Complete graph of publicly known software development: repos, commits, directories, files, and their relationships. Available as node+edge export.	Dependency and impact analysis at scale. Can support Dataset 3 (downstream impact chains) and Dataset 6 (cross-repo dependency context) via ecosystem-level call/import graphs. Very heavy; best used via targeted API or BigQuery queries rather than full ingestion.
GH Archive	Complete GitHub event stream since 2011 (PushEvents, PullRequestEvents, IssuesEvents, IssueCommentEvents, etc.), available as JSON via BigQuery or direct download.	Primary scraped training source. Mine issue→PR→fix chains at scale. Review comments often contain explicit missing-context signals (e.g., “this breaks because of X in module Y”). Feeds context-attributed failures, design rationale datasets and supports mental model construction via repo evolution.
deps.dev	Google’s dataset of dependency relationships across package ecosystems (npm, PyPI, Go, Maven, Cargo, etc.), available via BigQuery. Includes dependency graphs, versions, advisories, and licenses.	Feeds dependency context. Provides version graphs, breaking-change signals, and advisory data when indexing repo dependencies. High priority for the dependency index.
ARKS / EvoR-Bench	Four datasets covering two realistic RACG scenarios: frequently updated Python libraries and long-tail programming languages. Introduced alongside the ARKS/EvoR pipeline, which evolves both retrieval queries and a diverse knowledge base (documentation, web search, execution feedback, code snippets) iteratively rather than using a static single-source corpus. Evaluated with execution-based accuracy.	Evaluation of dynamic, multi-source retrieval for code generation. Directly relevant for testing whether retrieval pipelines can handle domain-shift scenarios (e.g., recently updated APIs or niche languages). The diverse “knowledge soup” design (docs + web + execution feedback) mirrors realistic agentic RAG setups. Useful for benchmarking retrievers under active, multi-round query refinement rather than one-shot retrieval. Medium priority.

Dataset	Description	Commentary
GitChameleon 2.0	328 manually authored Python code-completion problems, each conditioned on a specific library version (spanning 26 libraries from 2014–2023) and accompanied by executable unit tests. Designed to evaluate <i>version-conditioned generation</i> (VCG): whether a model can produce code that is functionally correct for a <i>particular</i> , potentially non-latest library version. Includes documentation references to support RAG evaluation, and supports self-debugging via visible tests.	Evaluation of version-aware RAG for code generation. Tests whether a retrieval system can surface the right version-specific documentation to resolve API incompatibilities—a highly practical scenario for production codebases pinned to older library versions. Execution-based evaluation ensures functional correctness rather than surface-level matching. Useful for assessing whether retrieved docs genuinely resolve version conflicts. Medium-high priority.
Mutation-induced failing patches	Generate via <code>mutmut</code> (or similar) on repos with tests. Inject mutations, run tests, retain failing mutants. Produces <code>(mutation.diff, failing_tests, original_code)</code> triples.	High-volume source for primary and secondary datasets. Controlled perturbations provide exact change provenance and traceable failure paths. Suitable for large-scale retrieval-ranker training (“what context would have prevented this mistake?”). Generation should run on repos already in the data lake.
Mutation-repair pairs with context budgets	For each mutated bug, collect an agent fix under limited context and under augmented context; compare outcomes.	Highest-value synthetic dataset. Directly captures the causal signal: did context X change the outcome from wrong to right? Expensive to generate (requires agent runs), but highest signal-to-noise ratio.
Documentation generation	Generate or extract docstrings and use them to train similarity between code semantics and natural language descriptions.	Secondary priority. Supports embedding fine-tuning to better align code and documentation (e.g., Milvus). Improves <code>get_dependency_context</code> and other NL-based retrieval. Can be deferred until semantic search becomes the bottleneck.
Closed issues + closing PRs + merge diffs	Scrape via the GitHub GraphQL API: issue text, discussion threads, linked PRs, review comments, and commit diffs. Produces <code>(task.description, discussion_context, fix_diff)</code> tuples.	Primary scraped dataset (high priority). Closest approximation to naturally occurring Dataset 1 instances. Review comments often contain explicit missing-context annotations (“this breaks X because Y”). Feeds retrieval ranking, mental model construction, and convention mining. Should be prioritized in the offline pipeline.

4 Data-processing pipelines

The data-processing pipeline is implemented as the execution plane of Lighthouse and is designed to continuously supply high-quality context to the MCP service while also producing durable datasets for evaluation and model-improvement work. Architecturally, the pipeline runs as long-lived [Temporal](#) workflows and applies a consistent stage model across online and offline workflows: ingest source data, clean and normalize it, transform it into retrieval/evaluation artifacts, and store outputs in a two-tier storage design. In this design, large raw and curated files are written to an S3-based data lake (bronze/silver/gold), while structured operational tables and queryable metadata are stored in Postgres. Retrieval-ready vectors are stored in Milvus.

4.1 Data Schemas

Lighthouse operates in two complementary pipeline modes. The online pipeline is request-driven and focuses on indexing freshness and low-latency readiness for MCP tool calls. The offline pipeline is schedule-driven and focuses on durable dataset curation for evaluation and retrieval-quality improvements: rolling scraped sources are ingested incrementally, synthetic datasets are refreshed on cadence, and version-pinned benchmark snapshots are treated as immutable to preserve reproducible baselines.

4.1.1 Online indexing/control schemas

IndexJobSchema

Field	Type	Notes
job_id	str	Unique indexing job identifier
workflow_id	str	Temporal workflow identifier
repo_id	str	Stable repository identifier
ref	str	Branch or tag context
status	IndexStatus	Job lifecycle state
stage	IndexStage	Current pipeline stage
progress_pct	int	Progress percentage (0–100)
error_code	str null	Machine-readable failure code
error_message	str null	Human-readable failure message
created_at	timestamp	Job creation time
updated_at	timestamp	Last status update time

Per-job execution progress, stage transitions, and failure details for runtime and offline indexing runs.

IndexStateSchema

Field	Type	Notes
repo_id	str	Stable repository identifier
ref	str	Branch or tag context
snapshot_sha	commit_sha	Indexed commit represented by this state
status	IndexStatus	Readiness/freshness state
active_job_id	str null	Current active job if indexing is in progress
stale_after	timestamp	Freshness threshold for this index
last_indexed_at	timestamp	Last successful index completion
updated_at	timestamp	Last state update time

Represents the latest readiness and freshness state for each (`repo_id`, `ref`) pair consumed by MCP.

4.1.2 Offline dataset schemas

RepoSnapshotSchema

Field	Type	Notes
repo_id	str	Stable repository identifier
ref	str	Branch or tag context
snapshot_sha	commit_sha	Point-in-time repository snapshot
source	str	Upstream source name
captured_at	timestamp	Ingestion timestamp

Stores reproducible point-in-time repository snapshots used to anchor downstream dataset records.

DatasetInstanceSchema

Field	Type	Notes
instance_id	str	Unique dataset instance identifier
task	str	Task/problem description
repo_id	str	Repository identifier
snapshot_sha	commit_sha	Snapshot associated with this instance
failure_type	str	Failure category label
failure_ref	str	Pointer to failing output/signal
corrected_diff_ref	str	Pointer to corrected change artifact
split	str	Dataset split (train/validation/test)
created_at	timestamp	Record creation time

Defines the canonical task, failure, and fix record used for training and evaluation datasets.

ContextLabelSchema

Field	Type	Notes
instance_id	str	Linked dataset instance
source_type	str	Context origin (repo/dependency/doc/history)
location	str	File span, URL, or commit reference
context_type	str	Context class (contract/caller/invariant/etc.)
relevance_reason	str	Why this context matters
created_at	timestamp	Record creation time

Captures structured attribution of missing non-local context for each dataset instance.

QualityMetricSchema

Field	Type	Notes
run_id	str	Linked pipeline run identifier
metric_name	str	Metric key/name
metric_value	float	Numeric metric value
metric_context	str	Scope/context label for interpretation
measured_at	timestamp	Measurement timestamp

Stores quality and completeness metrics used to monitor dataset integrity and drift.

PipelineRunSchema

Field	Type	Notes
run_id	str	Unique pipeline run identifier
workflow_type	str	Workflow family (offline/runtime/mental-model)
source_name	str	Dataset/source adapter name
status	str	Run lifecycle state
started_at	timestamp	Run start time
finished_at	timestamp null	Run completion time
records_in	int	Input row count
records_out	int	Output row count
error_code	str null	Failure code if run did not succeed

Provides run-level observability, auditability, and operational health tracking for pipeline executions.

4.2 Pipelines

4.2.1 Schedules and Use Cases

Pipeline	Schedule	Primary use case
Runtime indexing (RuntimeIndexWorkflow)	Event-driven (on MCP request)	Create or refresh repository context so MCP tool calls can return current results.
Retrieval pipeline (agent → MCP tool call)	Event-driven (per MCP request)	Retrieve relevant repository context (code, diffs, issues/PRs, docs) with freshness checks, reranking, and bounded results to ground the agent response.
Offline benchmark snapshot ingestion (OfflineDatasetWorkflow)	Manual or on new benchmark release	Import and normalize immutable benchmark snapshots for reproducible baselines.
Evaluation refresh jobs	Monthly or on snapshot update	Recompute fail-to-pass delta and regression-rate trendlines over pinned benchmark data.
Offline rolling scraped ingestion (OfflineDatasetWorkflow)	Daily incremental	Ingest issue to PR to diff chains with append/upsert semantics for continuously updated training/eval slices.
Offline synthetic dataset refresh (OfflineDatasetWorkflow)	Weekly	Regenerate synthetic mutation-repair datasets and derived features with versioned generators.
Offline targeted reprocessing (OfflineDatasetWorkflow)	Event-driven/manual	Reprocess impacted partitions only after schema updates, transformation fixes, or data correction events.

4.2.2 Pipeline Diagrams

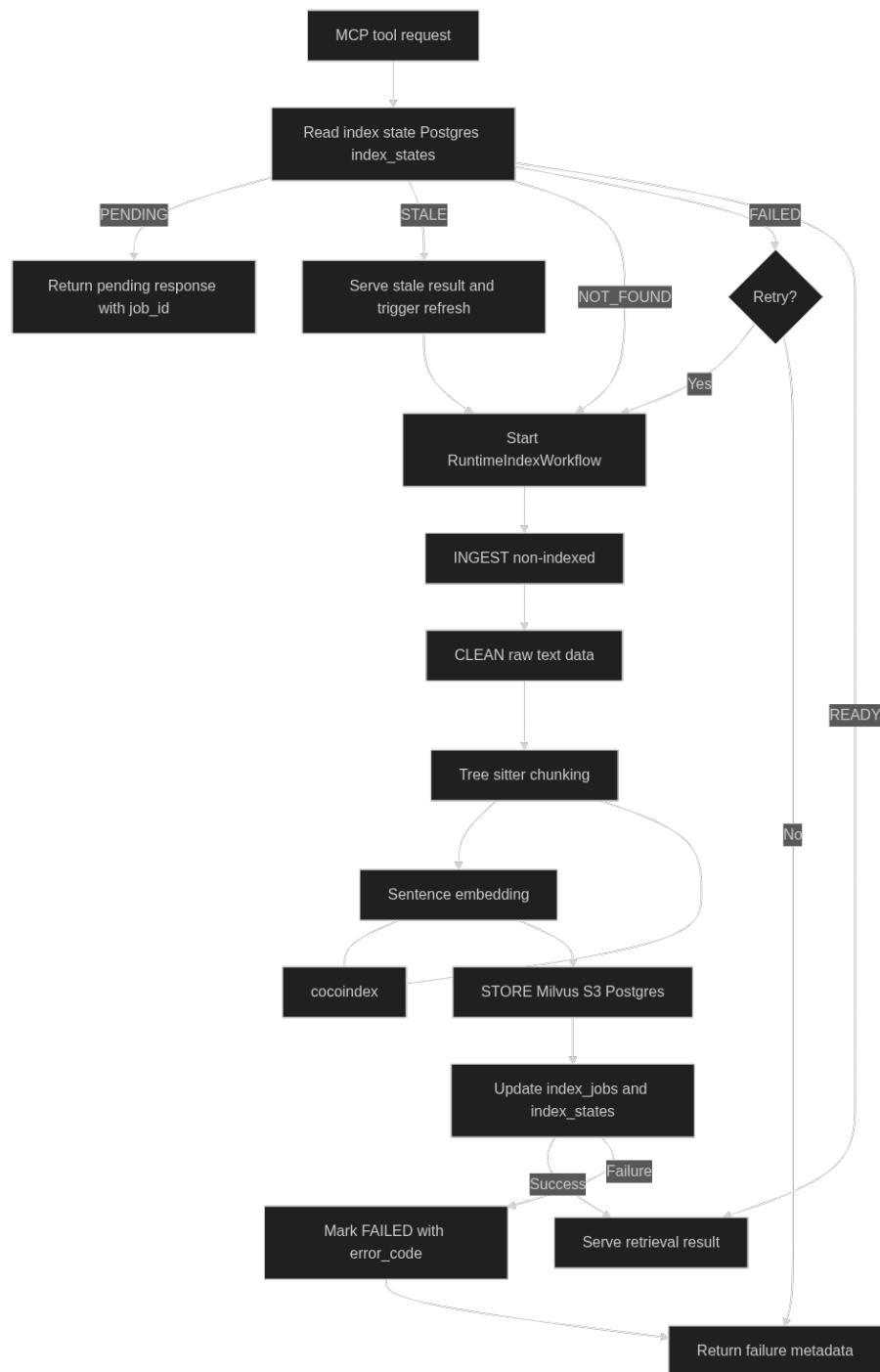


Figure 1: Runtime indexing workflow (RuntimeIndexWorkflow).

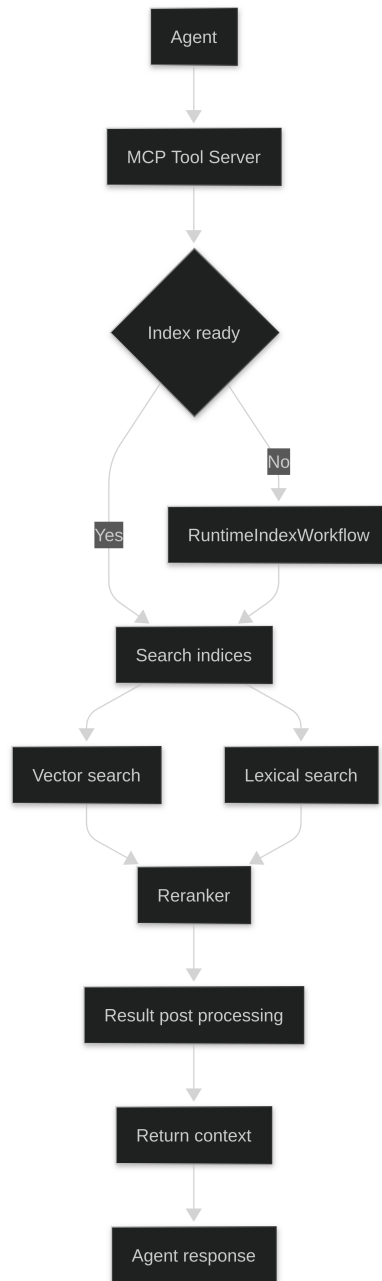


Figure 2: Agent retrieval workflow (`AgentRetrievalWorkflow`).

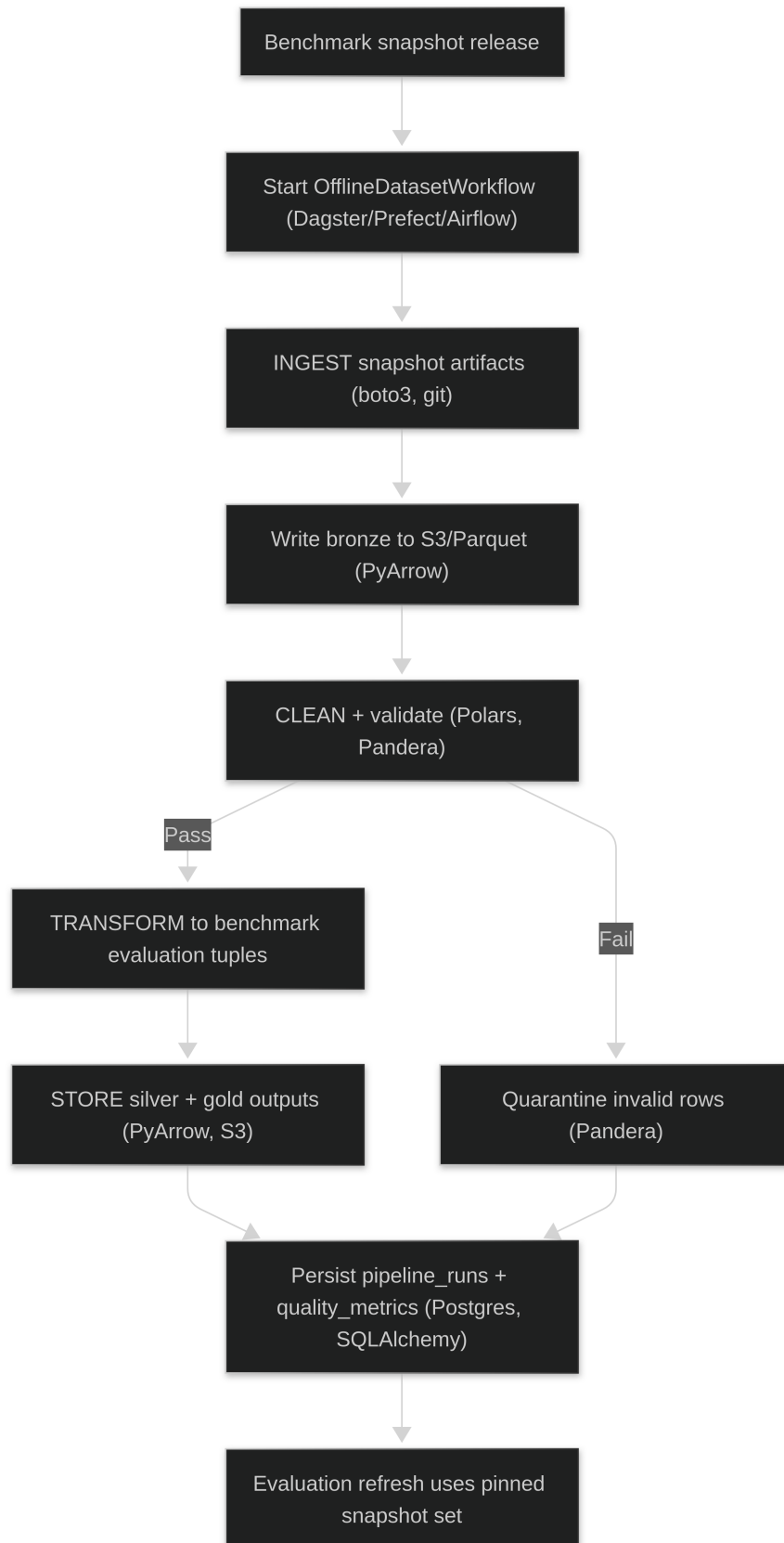


Figure 3: Offline benchmark snapshot workflow (OfflineDatasetWorkflow).

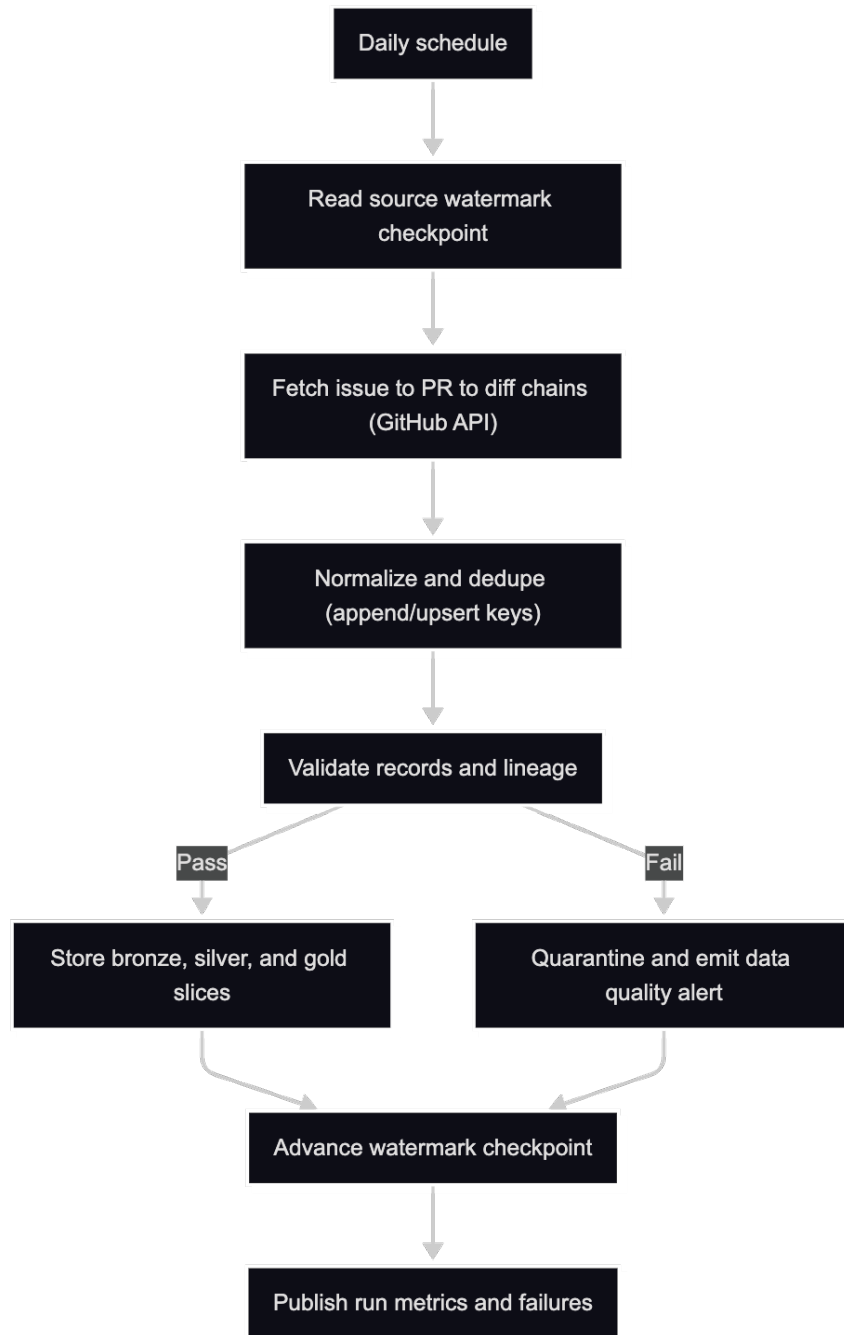


Figure 4: Offline rolling scraped workflow (OfflineDatasetWorkflow).

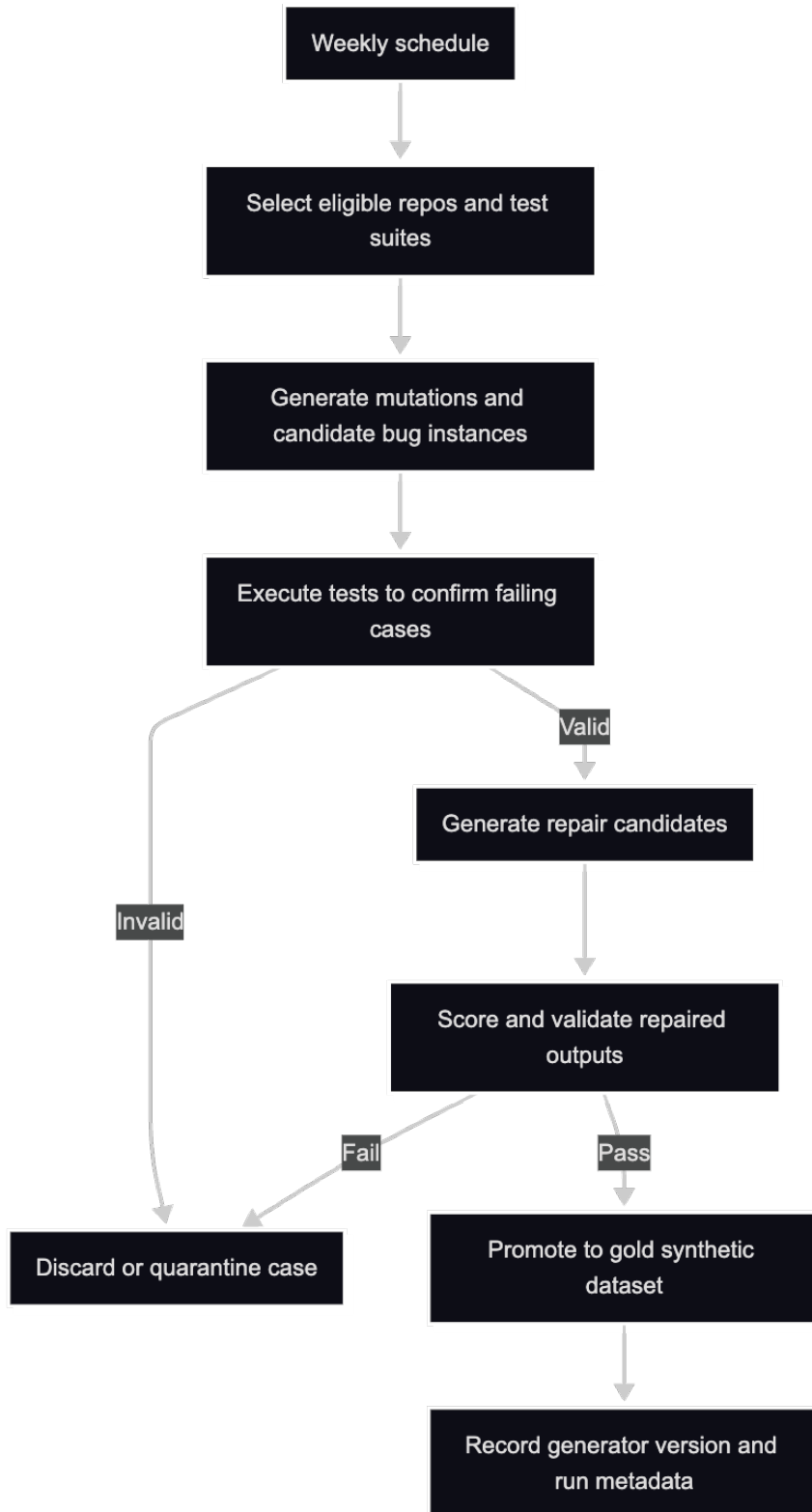


Figure 5: Offline synthetic refresh workflow (OfflineDatasetWorkflow).

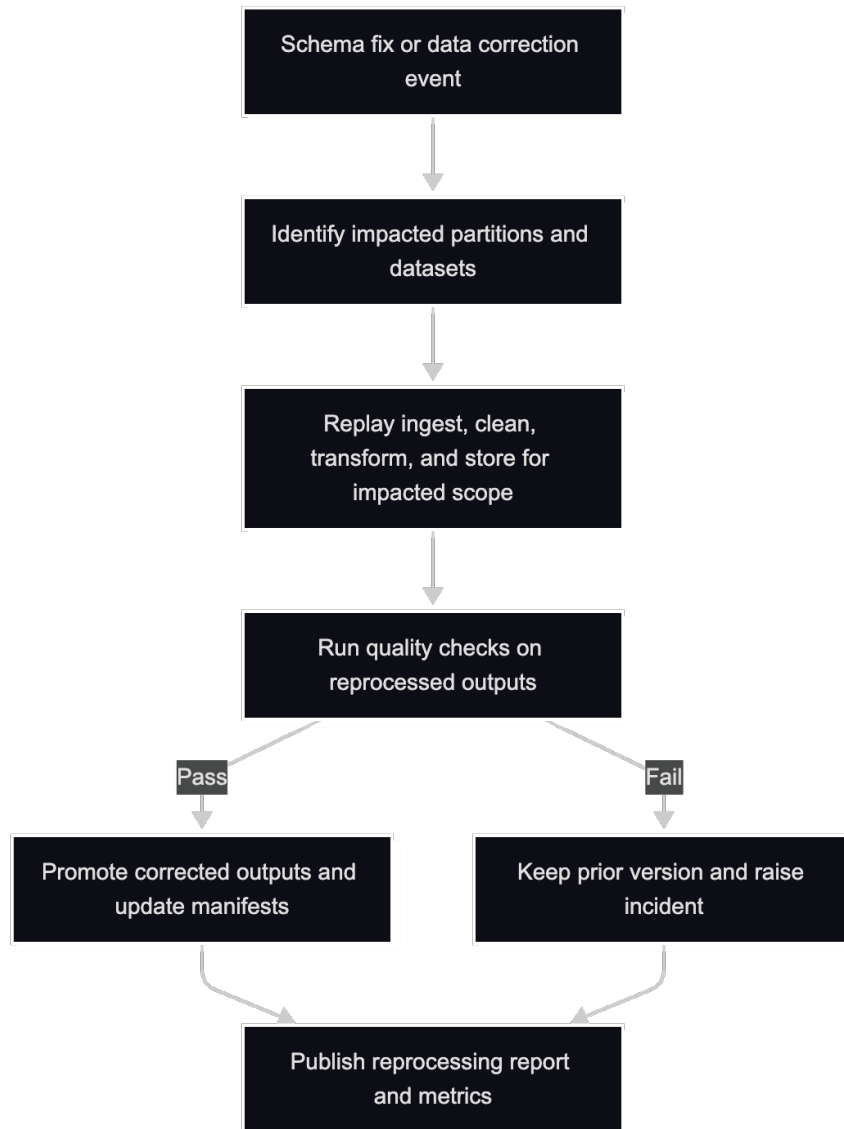


Figure 6: Offline targeted reprocessing workflow (OfflineDatasetWorkflow).

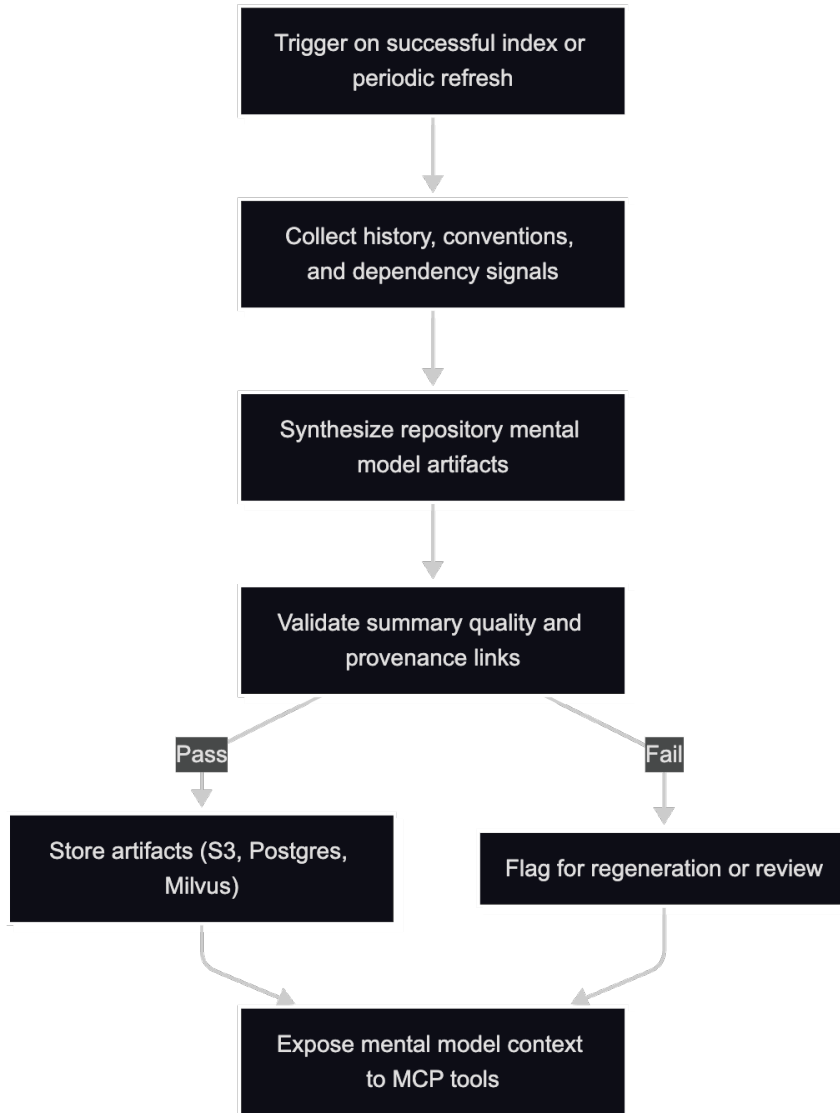


Figure 7: Mental model workflow (MentalModelWorkflow).

4.3 Future Roadmap

The first iteration established the end-to-end architecture, canonical schemas, and Temporal workflow structure, and it delivered a working online runtime-indexing pipeline plus benchmark snapshot ingestion for offline evaluation. The next iteration will focus on:

1. Add a rolling scraped offline pipeline for incremental issue to PR to diff ingestion. This workflow will ingest new chains on a recurring cadence using append/upsert semantics keyed by stable source identifiers, rather than full rebuilds. The first implementation will focus on deterministic dedupe, watermark-based progress tracking, and run-level observability so ingestion can be resumed safely after failures.
2. Add a synthetic offline pipeline for mutation-repair dataset refresh. This pipeline will generate controlled failing/repair examples on selected repositories with reliable test suites, then promote only validated outputs into gold datasets. Generator configuration, model version, and scoring settings will be versioned so synthetic data remains reproducible and comparable across runs.

3. Expand offline schemas with high-confidence context-label attribution (`missing_context`). The next schema revision will introduce explicit context-label tables and lineage links back to source artifacts and transforms. Initial labels will be weakly supervised and then calibrated through sampled review so downstream ranking/evaluation can account for label confidence.
4. Train and integrate a learned retrieval-ranking layer over gold datasets. We will start with an offline training/evaluation loop that compares learned reranking against current retrieval baselines on fail-to-pass and regression metrics. Once stable, the ranker will be introduced behind configuration flags so rollout can be incremental and reversible.

5 IaC Implementation

This section describes the Infrastructure-as-Code (IaC) implementation used to deploy the Lighthouse MVP execution plane on AWS. The implementation is written in Terraform and is organized into small, purpose-built modules to keep deployments straightforward, maintainable, and evolvable. The infrastructure supports (i) the MCP service (online request plane), (ii) the pipeline workers (offline and runtime execution), (iii) workflow orchestration (Temporal), (iv) vector retrieval (Milvus), and (v) durable storage layers (S3 data lake + Postgres operational metadata), matching the pipeline architecture above.

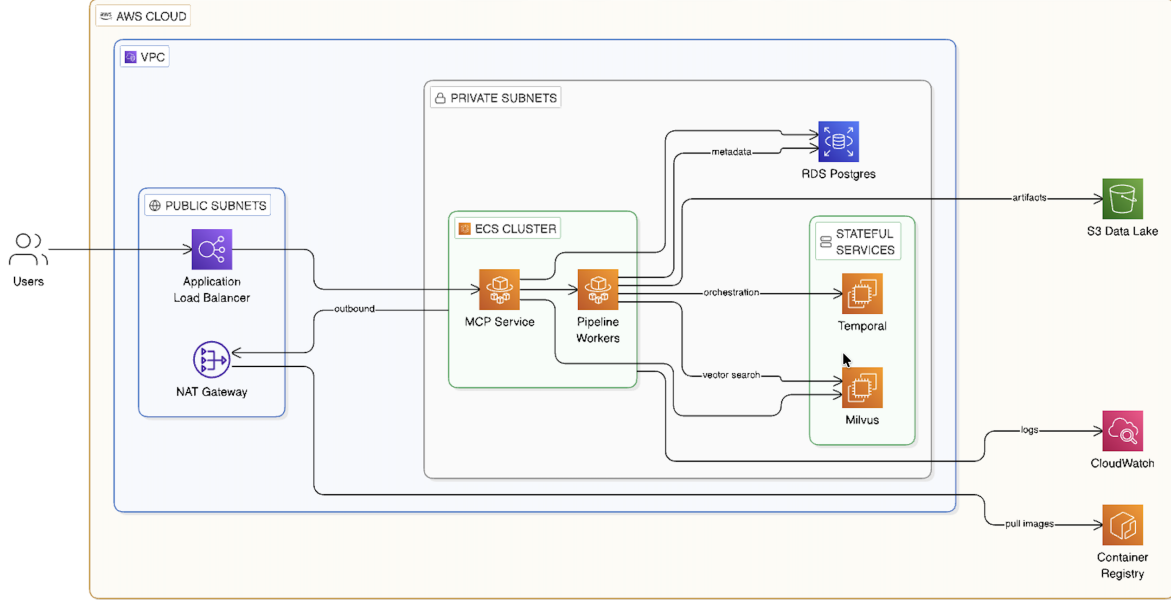


Figure 8: Infrastructure Diagram

5.1 IaC Approach and Repository Organization

We chose Terraform for three reasons: (1) the AWS provider ecosystem is mature, (2) a declarative plan/apply workflow makes environments reproducible, and (3) modular composition makes it easy to extend the MVP into production incrementally.

The Terraform project is split into a root module and four child modules that can be found in the `infra/` directory:

- `main.tf`, `variables.tf`, `outputs.tf`: root module wiring and environment-level configuration.
- `modules/networking`: VPC, subnets, routing, NAT, and optional VPC endpoints.
- `modules/storage`: S3 data lake bucket with encryption, versioning, and public access blocks.
- `modules/database`: RDS Postgres in private subnets and least-privilege security groups.
- `modules/compute`: ECS Fargate (MCP + worker), ALB ingress for MCP, IAM roles, logs, and self-hosted Temporal/Milvus on separate EC2 instances.

This module decomposition is intentionally aligned with the conceptual architecture: networking and storage primitives are independent; stateful databases are isolated; stateless services are deployed via ECS; and self-hosted stateful systems (Temporal and Milvus) live behind private subnets.

5.2 Infrastructure Components and Completeness (MVP)

The MVP infrastructure is “complete” in the sense that it provides all required services to run end-to-end indexing and offline workflows:

Networking (VPC, Subnets, Routing, NAT)

- A single VPC with DNS support enabled (required for service discovery).
- Public subnets for the Application Load Balancer (ALB) and NAT Gateway.
- Private subnets for ECS tasks, RDS, and EC2 stateful services (Temporal and Milvus).
- Separate route tables for public and private subnets to avoid accidental exposure.
- A NAT Gateway to enable outbound internet access for private workloads (e.g., pulling images from ECR, cloning repos, or reaching public documentation sources during ingestion).

Storage (S3 Data Lake)

- One S3 bucket acts as the data lake for pipeline artifacts and staged data (bronze/silver/gold).
- Server-side encryption (SSE) and versioning are enabled by default.
- Public access is blocked at the bucket level.

Operational Database (RDS Postgres)

- A Postgres instance is deployed in private subnets using an RDS subnet group.
- The RDS security group is configured for least-privilege access: inbound Postgres connections are allowed only from approved security groups (ECS tasks), rather than broad CIDR blocks.
- Production safeguards are enabled conditionally: deletion protection and longer backup retention in prod; lightweight backups in dev.

Compute Plane (ECS Fargate + ALB)

- An ECS cluster runs two services on Fargate:
 - **MCP service:** deployed as a private ECS service behind a public ALB.
 - **Pipeline/Temporal worker service:** deployed as private ECS tasks with no inbound exposure.
- CloudWatch log groups are created for MCP and worker logs.
- IAM roles are split between:
 - **Execution role:** required to pull images from ECR and emit logs.
 - **Task role:** application permissions (e.g., scoped access to the S3 data lake).

Self-Hosted Workflow and Vector Systems (EC2) Temporal and Milvus are self-hosted on separate EC2 instances in private subnets. This choice is driven by cost constraints: we have AWS credits available, but do not have credits for managed Temporal/Milvus offerings. Self-hosting allows us to remain within budget while still meeting our functional requirements.

5.3 Use of Scripts (Justification)

For stateful services (Temporal and Milvus), we use `user_data` to bootstrap docker and run docker-compose. This is a deliberate MVP tradeoff:

- **Why scripts here:** Terraform does not provide a native, first-class abstraction for installing and operating multi-container applications on EC2 beyond provisioning the instance itself. Using `user_data` keeps the deployment self-contained and reproducible without introducing additional orchestration layers (e.g., Kubernetes) prematurely.
- **Why not for everything:** All AWS primitives (VPC, subnets, ALB, ECS, IAM, RDS, S3) are managed declaratively in Terraform modules. Scripts are limited to service bootstrapping where Terraform is not the right tool for day-2 operations.
- **Upgrade path:** As the project matures, we can replace bootstrap scripts with a more structured approach (e.g., SSM documents, AMIs, or a dedicated configuration management step) while keeping the Terraform interface stable.

5.4 Security Controls

Security is enforced via network segmentation and least-privilege IAM:

- **Ingress minimization:** Only the ALB is public. MCP tasks allow inbound only from the ALB security group. Worker tasks have no inbound rules.
- **Database isolation:** RDS is not publicly accessible and allows inbound only from ECS task security groups.
- **Stateful isolation:** Temporal (7233) and Milvus (19530) are reachable only from inside the VPC; SSH is restricted to an admin CIDR when a key pair is configured.
- **S3 hardening:** encryption + versioning + public access blocks are enabled by default.

5.5 Alignment with Requirements

The IaC implementation is intentionally mapped to the architectural requirements:

- **Online runtime indexing and MCP tool calls:** The MCP service is deployed as a low-latency, horizontally scalable ECS service behind an ALB, matching the online request-driven pipeline described earlier.
- **Offline and scheduled pipelines:** Pipeline workers run as ECS services and execute ingestion, transformation, and indexing jobs; Temporal provides workflow orchestration and durable retries consistent with the staged pipeline model.
- **Two-tier storage design:** S3 holds large raw/curated artifacts (data lake), Postgres stores structured operational metadata and pipeline run state, and Milvus stores retrieval-ready vectors for context search.

5.6 Multiple Environments

The Terraform implementation supports multiple environments (`dev`, `staging`, `prod`) using environment-scoped naming and configuration:

- **Namespacing:** resource names include `<project>-<environment>` to prevent collisions.
- **Environment-specific safety toggles:**
 - S3 `force_destroy` enabled in `dev` for rapid iteration; disabled in `prod`.
 - RDS deletion protection and longer backup retention enabled in `prod`.

- Instance sizes and desired task counts can be varied by environment via `*.tfvars`.
- **Operational workflow:** environments can be deployed independently using separate variable files, for example:
 - `terraform apply -var-file=dev.tfvars`
 - `terraform apply -var-file=prod.tfvars`

6 Disaster Recovery Demonstration

Demo:

https://drive.google.com/file/d/1fJBwFgofLGQfy_2WBrokKrKQp3qT8qbf/view?usp=sharing

Appendix

A Aspirational Datasets: Schema Type Definitions

A.1 Dataset 1: Context-Attributed Agent Failures

AgentAttempt

Field	Type	
diff	Diff	incorrect patch
files_viewed	[str]	files inspected
searches_performed	[str]	issued queries

FailureSignal

Field	Type	
type	enum	test_failure type_error runtime_error semantic_bug convention_violation
message	str	diagnostic message
affected_locations	[{file,line}]	manifestation points

Context

Field	Type	
source	enum	same_repo dependency documentation history
file	str	path / URL / commit
span	(start,end)	line range
content	str	contextual excerpt
context_type	enum	contract caller type_def invariant convention rationale config doc
explanation	str	why required

A.2 Dataset 2: Interface Contracts and Cross-Boundary Specifications

Contract

Field	Type	
signature	str	type signature / prototype
preconditions	[str]	required inputs
postconditions	[str]	guarantees on return
invariants	[str]	preserved properties
implicit	[str]	unwritten assumptions
source_of_truth	enum	type_system docstring test_assertion usage_pattern unwritten

Consumer

Field	Type	
location	{repo,file,span}	call site
assumption_relied_on	str	contract dependency

KnownViolation

Field	Type	
diff	Diff	violating change
which_contract_broken	str	violated clause
failure_mode	str	manifestation type

A.3 Dataset 3: Downstream Impact Chains

CausalNode

Field	Type	
node	{file,function}	intermediate location
relationship	enum	calls inherits imports configures instantiates overrides
what_propagates	str	propagated change (e.g. return type, side effect)

A.4 Dataset 4: Architectural Conventions and Project Norms

Convention

Field	Type	
description	str	informal rule
category	enum	structural naming error_handling concurrency security testing other
evidence	[Evidence]	pattern-establishing locations
codified_in	str null	lint / ADR / style guide

Evidence

Field	Type	
file	str	file path
span	(start,end)	line range

Violation

Field	Type	
diff	Diff	violating change
what_was_violated	str	broken rule
corrected_diff	Diff	fix
detection_signal	enum	test_failure review_comment lint_error silent_bug

A.5 Dataset 5: Design Rationale and Temporal Context

HistoryEntry

Field	Type	
commit	sha	commit id
timestamp	datetime	commit time
author	str	author
diff	Diff	change introduced
message	str	commit message
linked_issue	str null	issue reference

Rationale

Field	Type	
summary	str	reason for current design
constraints	[str]	limiting factors
still_relevant	bool	validity flag
invalidated_by	str null	superseding event

NaiveRefactor

Field	Type	
diff	Diff	proposed change
why_incorrect	str	breaking reason
failure_signal	str	manifestation

A.6 Dataset 6: Cross-Repository Dependency Context

RequiredContext

Field	Type	
source_type	enum	source_code docstring changelog migration_guide issue_thread example
location	str	path / URL
content	str	relevant excerpt
relevance	str	why it matters

VersionSensitivity

Field	Type	
valid_for	version_range	safe versions
breaks_at	version null	breaking version
breaking_change	str null	behavioral change

MisuseExample

Field	Type	
diff	Diff	incorrect change
assumption_violated	str	broken assumption
failure_signal	str	manifestation